

UNIT 2: Arithmetic and Logic Unit (ALU)

Arithmetic and logic unit: Look ahead carries adders. Multiplication: Signed operand multiplication, Booths algorithm and array multiplier.

Division and logic operations. Floating point arithmetic operation, Arithmetic & logic unit design. IEEE Standard for Floating Point Numbers

◇ 1. Introduction

Arithmetic and Logic Unit (ALU) is the **heart of the CPU**.

It performs all **arithmetic** (add, subtract, multiply, divide) and **logic** (AND, OR, NOT, etc.) operations.

💡 Think of it as the **calculator and decision-maker** of the processor.

◇ 2. Arithmetic Operations

Arithmetic operations involve:

- **Addition**
- **Subtraction**
- **Multiplication**
- **Division**
- **Floating Point Arithmetic**

These are performed on **binary numbers** inside the CPU.

◇ 3. Adders

An **adder** is a digital circuit that performs **addition** of numbers.

(a) Half Adder

- Adds **two 1-bit numbers (A and B)**.
- Produces **Sum (S)** and **Carry (C)**.

Input	Output
A	B
0	0
0	1
1	0
1	1

Formula:

👉 $S = A \oplus B, C = A \cdot B$

(b) Full Adder

- Adds **three 1-bit numbers** (A, B, Carry-in).
- Produces **Sum** and **Carry-out**.

Formula:

👉 $S = A \oplus B \oplus C_{in}, C_{out} = AB + BC_{in} + AC_{in}$

◇ 4. Look-Ahead Carry Adder

◇ Problem:

In normal **Ripple Carry Adder**, each bit must wait for the carry from the previous bit → causes delay.

◇ Solution:

Look-Ahead Carry Adder (LCA) reduces delay by generating all carries **in advance**.

Logic:

- Generate (G) and Propagate (P) signals:
 - $G_i = A_i \cdot B_i$
 - $P_i = A_i + B_i$

Carry formulas:

- $C_1 = G_0 + P_0C_0$
- $C_2 = G_1 + P_1G_0 + P_1P_0C_0$
- and so on.

■ **Advantage:** Faster addition because carry is pre-calculated.

■ **Disadvantage:** More hardware (complex circuit).

💡 **Real-life analogy:** Like predicting exam results early by knowing each subject's marks in advance, instead of waiting one by one.

◇ 5. Multiplication

Binary multiplication is **repeated addition**.

Example:

$$\begin{aligned} & 101 \times 11 \\ = & 101 \times (1 + 10) \\ = & 101 + 1010 \\ = & 1111 \text{ (15 in decimal)} \end{aligned}$$

🔗 (a) Signed Operand Multiplication

Signed numbers can be represented using **2's complement**.

Rules:

Sign of Multiplicand	Sign of Multiplier	Sign of Product
+	+	+
+	−	−
−	+	−
−	−	+

🔗 (b) Booth's Algorithm

Used for **fast multiplication** of signed numbers in **2's complement form**.

Idea:

Instead of multiplying bit by bit, **Booth's algorithm** scans bits in pairs to reduce steps.

Rules:

Bit Pair	Operation
00	No operation
01	Add multiplicand to result
10	Subtract multiplicand
11	No operation
After each step	Arithmetic Right Shift (ARS)

Example:

Multiply $(-3) \times (5)$

→ Represent $-3 = 1101$ (4-bit), $5 = 0101$

Follow Booth's steps → Result = $1111\ 0001$ (i.e., -15)

■ **Advantage:** Fewer addition/subtraction operations.

■ **Disadvantage:** Complex logic circuit.

🔗 (c) Array Multiplier

- Simple **hardware-based** method for unsigned multiplication.
- Uses **AND gates and adders** arranged in an array.

💡 Example:

For 4-bit numbers, there are 4 rows of partial products → added using carry save adders.

■ **Advantage:** Very fast.

■ **Disadvantage:** Large hardware cost.

◇ 6. Division in Binary

Binary division = **repeated subtraction** (similar to long division).

Example:

$1100 \div 10$
 $\rightarrow 1100 / 10 = 110$ (i.e., $12 \div 2 = 6$)

Steps:

1. Compare divisor with part of dividend.
2. Subtract if possible (write 1 in quotient).
3. Bring down next bit.
4. Repeat until done.

■ Hardware Division Methods:

- Restoring Division
- Non-Restoring Division

(These are used in hardware design.)

◇ 7. Logic Operations

ALU performs **bitwise logical operations**:

Operation	Symbol	Example (A=1010, B=1100)	Result
AND	$A \cdot B$	1010 AND 1100	1000
OR	$A + B$	1010 OR 1100	1110
XOR	$A \oplus B$	1010 XOR 1100	0110
NOT	$\neg A$	NOT 1010	0101

Used for **masking, comparison, and control operations**.

◇ 8. Floating Point Arithmetic Operations

Used for **very large or very small numbers** (fractions, exponents).

Floating Point Representation

A number is represented as:

$$N = (-1)^s \times M \times 2^E$$

Where:

- **s** = Sign bit (0 = +, 1 = -)
- **M** = Mantissa (fraction part)
- **E** = Exponent (power of 2)

Example:

+6.75 $\rightarrow$ Binary = 110.11
 $\rightarrow$ Normalize: 1.1011×2^2
So: s=0, M=1.1011, E=2

Floating Point Arithmetic Steps

1. **Align Exponents** (make exponents equal)
2. **Add/Subtract Mantissas**
3. **Normalize Result** (adjust exponent and mantissa)
4. **Round & Store**

 **Advantage:** Large range of values.

 **Disadvantage:** Less precision (rounding error).

◇ 9. IEEE Standard for Floating Point Numbers

IEEE 754 is the **most common standard** for floating point representation.

Formats:

Type	Total Bits	Sign	Exponent	Mantissa
Single Precision	32	1	8	23
Double Precision	64	1	11	52

Example:

Represent -5.75 in IEEE 754 (single precision)

1. Decimal $\rightarrow$ Binary = $101.11 = 1.0111 \times 2^2$
2. Sign = 1 (negative)
3. Exponent = $2 + 127 = 129 \rightarrow$ Binary: 10000001
4. Mantissa = 01110000000000000000000
5. Final = **1 10000001 011100000000000000000000**

◇ 10. Arithmetic & Logic Unit Design

An **ALU** combines:

- **Arithmetic Circuit** (Add, Subtract, Multiply, Divide)
- **Logic Circuit** (AND, OR, NOT, XOR)
- **Control Lines** (decide which operation to perform)

Example Operation Selection Table:

Control Input	Operation
---------------	-----------

000	AND
001	OR
010	ADD
011	SUBTRACT
100	XOR
101	COMPARE

 Real-life analogy:

ALU is like a multi-function calculator that performs various math and logic operations based on control signals.

Summary of Unit 2

Topic	Key Idea
Adder Types	Half, Full, Look-Ahead
Multiplication	Booth's Algorithm, Array Multiplier
Division	Restoring / Non-Restoring
Logic Operations	AND, OR, NOT, XOR
Floating Point	IEEE 754, Mantissa + Exponent
ALU Design	Combines arithmetic and logical circuits

Important Difference Tables

1. Ripple Carry Adder vs Look-Ahead Carry Adder

Basis	Ripple Carry	Look-Ahead Carry
Speed	Slow	Fast
Carry Generation	Sequential	Parallel
Hardware	Simple	Complex
Delay	Increases with bit size	Fixed, minimal
Use	Small circuits	High-speed CPUs

2. Booth's Algorithm vs Array Multiplier

Basis	Booth's Algorithm	Array Multiplier
Type	Sequential	Parallel
Use	Signed numbers	Unsigned numbers
Speed	Slower	Faster
Complexity	Moderate	High (more hardware)
Efficiency	Reduces steps	Generates all partial products

3. Fixed Point vs Floating Point

Basis	Fixed Point	Floating Point
Range	Small	Very large
Precision	High	Lower (rounding error)
Representation	Integer or fraction	Mantissa + Exponent
Hardware	Simple	Complex
Use	Embedded devices	Scientific computation

4. Arithmetic vs Logic Operation

Basis	Arithmetic Operation	Logic Operation
Type	Mathematical	Bitwise
Examples	Add, Sub, Mul, Div	AND, OR, NOT, XOR
Output	Numeric	Logical (1 or 0)
Use	Calculation	Decision/control

1. Half Adder vs Full Adder

Basis	Half Adder	Full Adder
Inputs	2 (A, B)	3 (A, B, Carry-in)
Outputs	Sum, Carry	Sum, Carry
Carry Input	Not used	Used
Logic Used	XOR and AND	XOR, AND, OR
Example	Adds 1-bit numbers	Adds 3-bit numbers (with carry)

2. Ripple Carry Adder vs Look-Ahead Carry Adder

Basis	Ripple Carry Adder	Look-Ahead Carry Adder
Carry Generation	Sequential (bit by bit)	Parallel (predicts carry)
Speed	Slow	Fast
Hardware	Simple	Complex
Delay	Increases with bit size	Constant
Used In	Simple processors	High-speed CPUs

3. Signed vs Unsigned Numbers

Basis	Unsigned Numbers	Signed Numbers
Sign Bit	No sign bit	One bit for sign (0=+, 1=-)
Range (for 8-bit)	0 to 255	-128 to +127

Use	Quantities that can't be negative	Values that can be $\pm$
Example	11111111 = 255	11111111 = -1 (in 2's complement)

4. Booth's Algorithm vs Array Multiplier

Basis	Booth's Algorithm	Array Multiplier
Type	Sequential	Parallel
Speed	Slower	Faster
Use	Signed numbers	Unsigned numbers
Hardware	Simple	More complex
Efficiency	Reduces steps by skipping zeros	Generates all partial products

5. Fixed Point vs Floating Point Representation

Basis	Fixed Point	Floating Point
Form	Integer or simple fraction	Mantissa $\times 2^{\text{Exponent}}$
Range	Small	Very large
Precision	High (exact)	Limited (rounding error)
Speed	Fast	Slower
Used In	Embedded systems	Scientific & graphics computing

6. Restoring Division vs Non-Restoring Division

Basis	Restoring Division	Non-Restoring Division
Operation	Restores partial remainder when negative	Avoids restoring by alternating add/subtract
Speed	Slower	Faster
Hardware	Simpler	More complex control
Steps	More	Fewer
Use	Easy to implement	Efficient processors

7. Arithmetic Operations vs Logical Operations

Basis	Arithmetic	Logical
Meaning	Mathematical calculations	Bitwise manipulation
Examples	ADD, SUB, MUL, DIV	AND, OR, NOT, XOR
Output	Numeric value	0 or 1 (True/False)
Use	Computation tasks	Decision-making, masking

8. Fixed Point Addition vs Floating Point Addition

Basis	Fixed Point Addition	Floating Point Addition
Exponents	Not used	Must be aligned
Steps	Simple binary addition	Align → Add → Normalize
Speed	Faster	Slower
Hardware	Simple adder	Complex (needs normalizer)
Use	Integer operations	Scientific operations

9. IEEE 754 Single Precision vs Double Precision

Basis	Single Precision	Double Precision
Total Bits	32	64
Sign Bit	1	1
Exponent Bits	8	11
Mantissa Bits	23	52
Precision	Lower	Higher
Storage	Less memory	More memory

10. Arithmetic Circuit vs Logic Circuit

Basis	Arithmetic Circuit	Logic Circuit
Purpose	Performs math operations	Performs logical (bitwise) operations
Examples	Adders, Subtractors	AND, OR, XOR
Input Type	Numeric binary data	Boolean binary data
Output	Numeric result	Logic result (0/1)
Part of	ALU	ALU

Notes by - [GKM](#)